AI Engineering and Claude Architect · Part IV, Production

AI Production Engineering, Lesson 5

Cost Engineering and Semantic Caching

The semantic cache hit 40% of the time, and the team celebrated. It also told a customer that the refund rule for orders over $50 applied to orders under $50, because those two questions look almost identical.

Module	AI10 of 13
Lesson	5 of 7
Prerequisites	AI03 L5, AI10 L2 and L4, AI04 L1
Exam relevance	Batch is TS 4.5. Semantic caching isn't in the guide
Reading time	~30 minutes

Contents

1. Cost Engineering and Semantic Caching

Lesson 5 of 7 · AI Production Engineering

Cost Engineering and Semantic Caching

Key idea

Work through levers in order: first those that cost no quality, then those that do. Prompt caching, trimming input and batching are close to free. Changing model or effort is a trade-off that needs evals. A semantic cache returns an old answer to a different question, so its threshold has to be set by the cost of being wrong, not by its hit rate.

> *Before: prompt caching mechanics, cost per task, usage fields, embeddings. Teaches: ordering cost levers, cache economics and traffic shape, batching, budgets, attribution, and semantic caches and their failures. Used by: L7.*

The hook: two questions, one answer

A semantic cache embeds each question, and returns a stored answer if a past question is close enough. Here's what that looks like, using a **toy embedding (hashed character 3-grams) on pairs I invented**, so its scores don't represent a real embedding model. The shape of the problem is real. Output from `CostLab.java`:

```
Toy semantic cache, on invented pairs:
  0.830 same  How do I reset my password?          | How can I reset my password?
  1.000 same  How do I reset my password?          | how do i reset my password
  0.895 same  What is your refund policy?          | What's your refund policy?
  0.453 same  What are your opening hours?         | When are you open?
  0.076 same  Can I get my money back?             | What is your refund policy?
  0.814 same  Where is my order?                   | Where's my order?
  0.660 DIFF  How do I cancel my order?            | How do I cancel my subscription?
  0.874 DIFF  Refund policy for orders over $50    | Refund policy for orders under $50
  0.907 DIFF  Can I return this after 30 days?     | Can I return this after 90 days?
  0.732 DIFF  Do you ship to Canada?               | Do you ship to Ghana?
  0.702 DIFF  Is the premium plan refundable?      | Is the basic plan refundable?
  0.727 DIFF  How do I delete my account?          | How do I reset my account?

  threshold 0.95: useful hits 1 of 6, wrong answers 0 of 6
  threshold 0.85: useful hits 2 of 6, wrong answers 2 of 6
  threshold 0.75: useful hits 4 of 6, wrong answers 2 of 6
  threshold 0.65: useful hits 4 of 6, wrong answers 6 of 6
  threshold 0.50: useful hits 4 of 6, wrong answers 6 of 6
```

"30 days" and "90 days" scored **higher** than a correct paraphrase. A trained embedding model would do better on meaning, as AI04 L1 showed, but negation, numbers and small changes of a word are hard cases for any similarity measure. **Similar questions can have different answers, and similarity can't see that.**

Intuition: the secretary's folder

A busy secretary keeps a folder of past replies. If someone asks exactly what was asked before, reusing the reply is smart. If someone asks something that merely sounds like it, reusing it is how a client gets told the wrong contract terms.

Now, the cheaper way to help that secretary is a pre-printed page of standing facts, so they don't need to reread the manual each time. That's prompt caching, and it never gives a wrong answer. It only saves reading.

Mechanism

Levers, in order

The bundled API reference's cost guide orders levers by a principle: cost per **completed task**, and free wins before trade-offs. This table is my summary of it.

Order	Lever	Saves	Risk to quality
1	**Prompt caching**	Up to about 90% or more of cached input	None
2	**Trim input**	Everything downstream	Low, if you test
3	**Agent-loop hygiene**	Compounding context	Low
4	**Output tokens**	Output costs most per token	Low to moderate
5	**Batch**	50%	None, if nobody's waiting
6	**Effort and budgets**	Varies	**A real trade-off**
7	**Model selection**	Large	**A real trade-off**

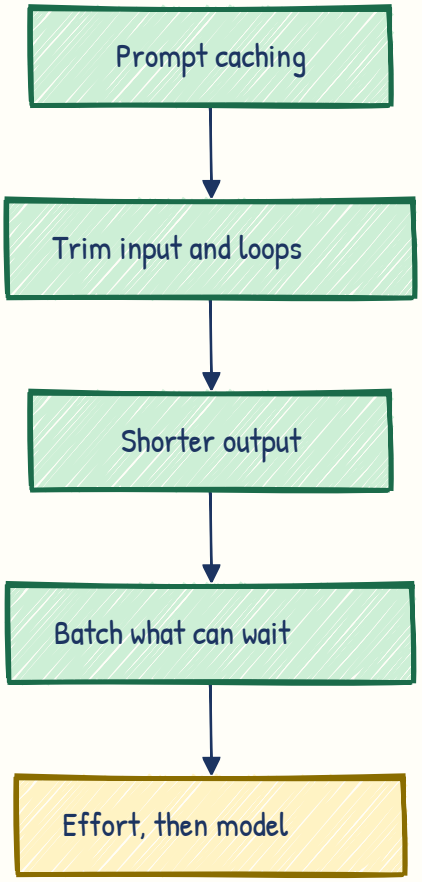


Figure 1. Free wins first. Trade-offs come last, and only after evals, because they can quietly lower quality.

The instinct is to begin with lever 7. **Changing model is the last lever, since it's the only big one that can quietly reduce quality.** And change one lever at a time, then measure, or you can't tell which one helped.

Lever 1: prompt caching

AI03 L5 taught the mechanics. Now the economics, on Sonnet 5.5. A 5-minute cache write costs 1.25 times input. Reads are priced per model: the pricing page says 0.05 times input on Opus 5.5 and Sonnet 5.5, and 0.1 on most others.

```
10,000-token prefix, 200 new input, 300 output, on claude-sonnet-5-5:
    1 calls within the TTL: without $  0.0234, with $  0.0284, saving -21.4%
    2 calls within the TTL: without $  0.0468, with $  0.0328, saving  29.9%
    3 calls within the TTL: without $  0.0702, with $  0.0372, saving  47.0%
   10 calls within the TTL: without $  0.2340, with $  0.0680, saving  70.9%
  100 calls within the TTL: without $  2.3400, with $  0.4640, saving  80.2%
```

It pays from the second call. One call alone costs more, so a one-shot job with a long unique prompt shouldn't cache.

Traffic shape decides the hit rate. Reads refresh the entry, so what matters is the gap between calls:

```
TTL walk with a 300-second TTL, where reads refresh the entry:
  one call every  30 s:  999 hits of 1000
  one call every 120 s:  999 hits of 1000
  one call every 280 s:  999 hits of 1000
  one call every 400 s:    0 hits of 1000
  one call every 900 s:    0 hits of 1000
```

It's a cliff. A feature used every six minutes never hits a 5-minute cache, and pays the write premium each time. Options: the 1-hour TTL at double the input price, or no cache. The bundled guide suggests this TTL walk over your real timestamps as a way of estimating hit rate.

What breaks it. Anything changing the prefix: a timestamp in the system prompt, reordered tools, or a per-user line placed before the shared text. AI03 L5 lists these, and L4's trace showed one. Verify from `usage`, and don't rely on reading the code.

Lever 2: trim input

Technique	Effect
Retrieve less, and better (AI05)	Fewer chunks per question
Trim tool results to needed fields (AI06)	Less in every later turn
Load tools or knowledge on demand	Smaller base prompt
Remove stale prompt text	Saves on every call

Agents compound. Every tool result stays in the history and is re-sent every turn, so a 5,000-token result in a 20-turn run is paid for repeatedly. Trimming early pays more than it looks. Use token counting before sending when it matters, as that's a free endpoint for estimating.

Lever 4: output

Output costs five times input here, per token. Ask for short answers and structured fields. Don't confuse this with lowering `max_tokens`, which is a safety cap: a cap that's too low gives truncated answers, and a retry costs more than what you saved.

Lever 5: Batch

From AI03 L5 and TS 4.5: half price, up to 24 hours, no latency guarantee, and no multi-turn tool use within a request. Correlate results by `custom_id`. Sonnet 5.5 in Batch is $1 and $5 per million tokens.

Good fit	Bad fit
Overnight reports, evals, backfills	A blocking pre-merge check
Document processing with a deadline of hours	A user waiting
Eval suites (L3)	Multi-step tool loops, unless flattened first

Data handling: Batch has 29-day retention, and isn't eligible for ZDR (AI09 L4).

Levers 6 and 7: trade-offs

L2 covered routing and cost per task. Effort (AI03 L4) is the same type of decision: lower effort is cheaper, and it needs evals. Fast mode costs more, at $8 and $40 per million on Opus 5.5, for speed. US-only inference with `inference_geo` is priced at 1.1 times.

Budgets and attribution

You can't reduce what you can't attribute. Invented traffic, with real prices:

```
Attribution:
  acme/chat      calls  900  spend $   4.68  per success $0.0052
  acme/report    calls  100  spend $  20.00  per success $0.2667
  globex/chat    calls  400  spend $   6.64  per success $0.0166
```

Two findings that a total can't show. `acme/report` is 7% of calls and most of the spend, with a quarter of runs failing. And `globex/chat` costs three times as much per message as `acme/chat` for the same work, because its setup isn't using the cache. Each suggests a specific fix.

Control	Where	Effect
Tier spend cap	Provider	Hard stop, with a 429 (L1)
Your spend limit	Provider	Hard stop, with a 400
Workspace limits	Provider	Isolates workloads
Per-tenant budget	Gateway	Your product's fair use (L7)
Per-run token limit	Agent	Stops runaway loops (AI08 L3)
Alert on spend rate	Monitoring	Hears about it first

Semantic caching

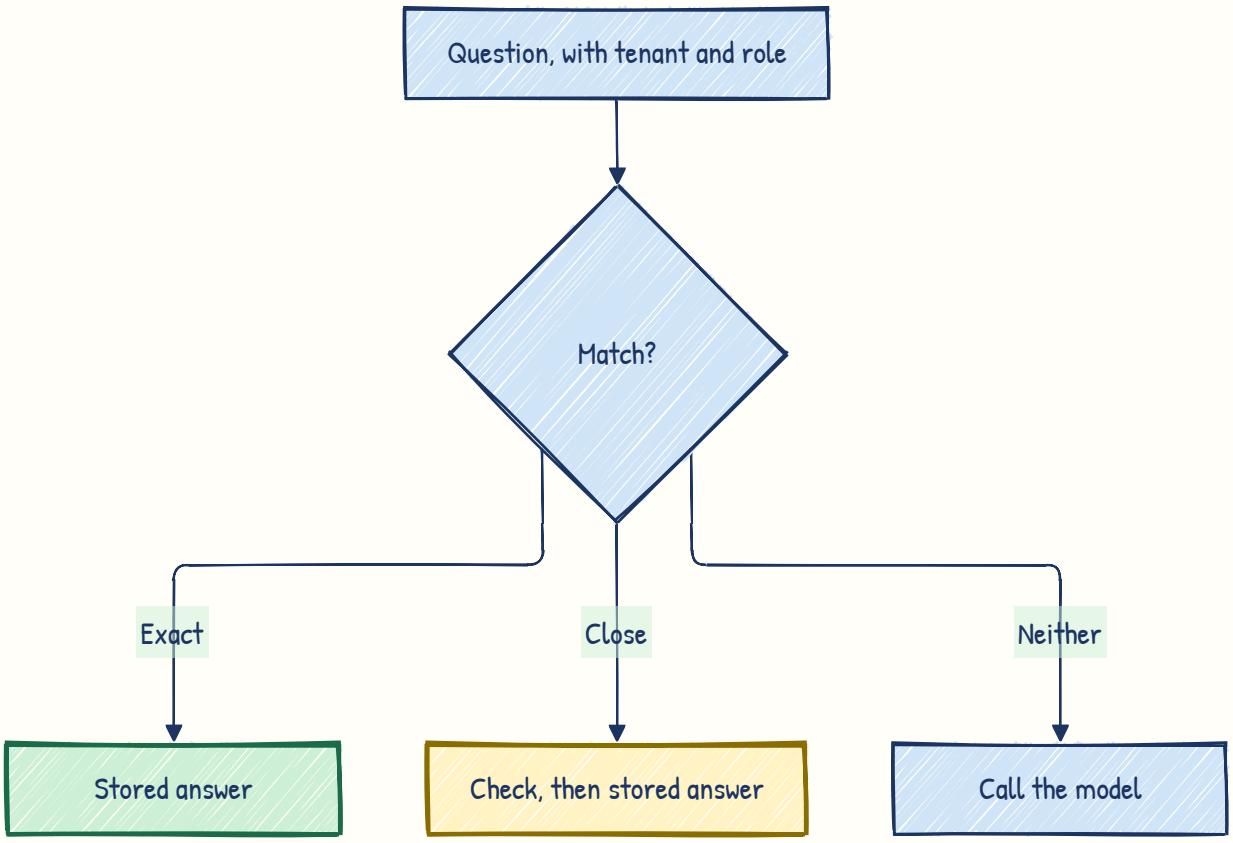


Figure 2. The lookup adds cost and risk on every request. Exact matches can't return a different question's answer, and close ones can.

Is it worth having? Compare what it saves with what a wrong answer costs. With an illustrative $5 per wrong answer:

```
Per million questions, call $0.0130, wrong answer $5.00:
  hit rate 10%, wrong share of hits  0.01%: saves $  1,300, harm $      50, net $    1,250
  hit rate 30%, wrong share of hits  0.01%: saves $  3,900, harm $     150, net $    3,750
  hit rate 30%, wrong share of hits  0.10%: saves $  3,900, harm $   1,500, net $    2,400
  hit rate 30%, wrong share of hits  1.00%: saves $  3,900, harm $  15,000, net $  -11,100
  hit rate 60%, wrong share of hits  1.00%: saves $  7,800, harm $  30,000, net $  -22,200
```

At a 1% wrong-answer share, a **higher** hit rate loses **more** money. Each call only costs about a cent, so almost any real cost of a wrong answer outweighs many saved calls. That's why I'd treat semantic caching as a specialist tool.

Where it fits, and where it doesn't (engineering recommendation):

Fits	Doesn't fit
FAQ with a small set of intents	Questions that depend on the person's data
Public, slow-changing content	Answers with numbers, dates or eligibility
Low cost of error	Medical, legal, or financial advice
Latency matters a lot	Multi-turn conversations, where context changes the meaning
	Anything that calls tools

An often better design: cache by intent. Classify the question into a known intent with a cheap model, then return a human-approved answer for that intent. This is safer, because the classifier chooses among reviewed answers, and it can abstain.

Rules if you build one:

Rule	Why
Tenant and roles in the key and filter	AI09 L4's cross-tenant leak
Include model, prompt and source versions	A stale answer survives a fix otherwise
Set the threshold with labelled pairs	As with L3, on your data, and include hard negatives
Invalidate when sources change	Refund policy changes, and the cache doesn't know
Set a TTL	Limits the damage from anything else
Check entities	Different numbers or names mean no hit
Sample hits for review	Wrong answers don't produce errors
Don't cache low-quality answers	Or one bad answer gets served repeatedly

Count the whole cost. The lookup needs an embedding and a vector search on every request, hit or miss. If hits are rare, the cache adds latency and cost overall.

Why the obvious fixes fall short

Obvious idea	Why it falls short
Switch to a cheaper model first	Risks quality, before free wins are used
Semantic cache with a high hit rate target	At 1% wrong, more hits lose more
Cache every prompt	One-shot calls cost 21% more
Lower `max_tokens` to save	Truncation, then a costlier retry
Compress prompts aggressively	Quality loss, and possibly no cache benefit
Summarize by the total bill	Hides which feature's cost is the problem

How Anthropic solved it

Mechanism	What's published	Source
Prompt caching	Prefix reuse, TTLs and pricing	Prompt caching docs, AI03 L5
Batch	Half price, up to 24 hours	Batch docs, TS 4.5
Lever order	Free wins before trade-offs	Bundled cost guide
Effort	Trade cost and quality	Effort docs, AI03 L4
Token counting	Estimates before sending	Token counting docs
Reports	Usage and Cost Admin API	Admin docs

How the field approaches semantic caching. Anthropic's docs that I read don't recommend it, and it's a field practice. Several libraries and gateway products offer it, and I haven't evaluated any. The hard-negative problem above is why I'm cautious.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: one model, no caching, and a monthly invoice.

Improved: prompt caching, attribution, and Batch for background work. New problem: savings decay, because each prompt change or new feature can silently undo them.

Production: make cost a measured, gated property.

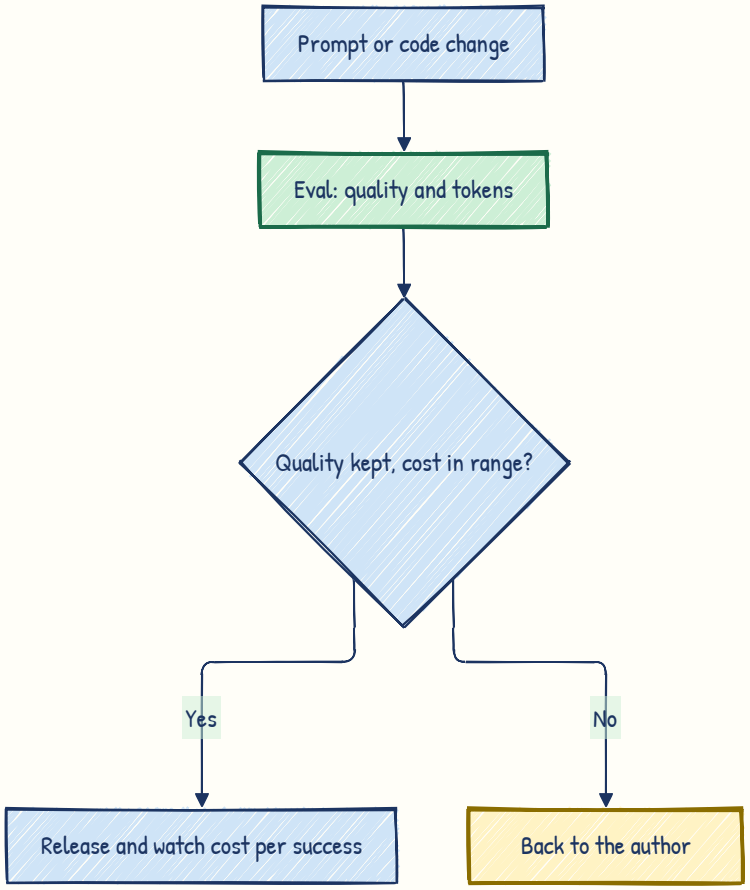


Figure 3. Cost is checked before release and watched after it. A change that breaks caching fails like any other regression.

Component	Why it exists	How it communicates
Usage pipeline	Cost per tenant, feature, and success	Spans and gateway records
Cost check in CI	Catches token growth and a broken prefix	Eval harness compares `usage`
Budget controls	Stops runaway spend	Gateway, provider limits
Batch router	Sends deferrable work to half price	Queue
Exact cache	Safe for identical requests	Key-value store
Semantic or intent cache	Only where justified	Vector store and reviewed answers

Failures:

Failure	How you'd notice	What to do
Prefix broken by a release	Hit ratio falls	Fail in CI. Roll back
Wrong cached answers	Sampled review, complaints	Raise the threshold, remove the entry, or use intents
Stale answer after a policy change	Complaints	Version keys by source, and invalidate
Cross-tenant cached answer	Two-tenant test	Scope keys
Runaway agent	Spend alert	Per-run limits
Batch misses a deadline	SLA tracking	Up to 24 hours is the guarantee, so plan margin
Savings make quality slip	Evals by segment	Revert the lever

Java

A cost is only correct when each field has its own price:

```
double with = new Usage(suffix, prefix, 0, output).cost(price)
        + (calls - 1) * new Usage(suffix, 0, prefix, output).cost(price);
```

The first call writes the prefix to cache, and later ones read it. That write premium is why one call loses money.

The cache's economics come down to one comparison:

```
double saved = hits * callCost;
double harm = wrong * wrongCost;
```

If you can't estimate `wrongCost`, you aren't ready to ship the cache.

Trade-offs

Decision	Benefit	Cost
Prompt caching	Large and safe	Prefix discipline. Write premium when hits are rare
1-hour TTL	Covers gaps	Double-price writes
Batch	Half price	Up to 24 hours, and retention rules
Semantic cache	Latency and some cost	Wrong answers that look right
Intent cache	Reviewed answers	Upkeep, and limited coverage
Smaller model	Large savings	Needs evals

When not to optimize: if the bill is smaller than an engineer's time to change it, leave it. Check that you have caching on, then return to the product.

Common mistakes

Reporting hit rate as success. A wrong cached answer counts as a hit. Measure wrong-answer share by sampling.

Common mistakes

Changing several levers at once. You can't attribute the saving, or the quality drop.

What to say in an interview

"I'd measure cost per completed task, by feature and tenant, and work through levers in order: caching, input, loop hygiene, output, Batch, and only then effort and model, which need evals. For a semantic cache, I'd compare savings with the cost of a wrong answer, since a call costs about a cent. I'd usually prefer an exact cache, or intent classification into reviewed answers, with tenant scope in any key."

Certification lens

How this lesson maps to CCAR-F

TS 4.5 covers Batch: 50% cheaper, up to 24 hours, no latency SLA, no multi-turn tools, and `custom_id` to correlate. **Prompt caching** appears through context management. **Semantic caching and gateway budgets aren't in the guide.**

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Code Review in CI · Medium. A team has a blocking pre-merge check and an overnight technical debt report, and wants to cut costs with the Batch API. What's the right use?

A. Move both to Batch, since it's cheaper

B. Use Batch for the overnight report, and keep the pre-merge check synchronous

C. Use Batch for the pre-merge check, and poll until it finishes

D. Don't use Batch, as it doesn't guarantee completion order

A **B** (TS 4.5). Batch has no latency SLA and can take up to 24 hours, so a blocking check can't wait for it. An overnight report can. A and C block developers, and D is a misreading, since `custom_id` correlates results.

Q Q2 · A feature calls the model every seven minutes with a large system prompt marked for 5-minute caching. Cost went up after caching was added. Why, and what are the options?

A Each call falls outside the 5-minute TTL, so it never reads the cache, and pays the 1.25x write price each time. Options: use the 1-hour TTL, if the write premium is repaid by reads, or remove caching for this feature, or shorten the prompt.

Q Q3 · When would you choose intent classification into approved answers over a semantic cache?

A When a wrong answer is costly, when there's a small set of intents, and when you can review the answers. The classifier chooses among vetted replies, and can abstain. The cost is writing and maintaining those answers, and coverage limited to the known intents.

Revision - Lesson 5

→ Work the levers in order: caching, input, loop hygiene, output, Batch, then effort and model.

→ Optimize cost per completed task. Change one lever at a time.

→ Caching pays from the second call within the TTL. One call alone costs more.

→ Gaps longer than the TTL give no hits at all, so traffic shape decides.

→ Cache reads are priced per model. Check the pricing page.

→ Agent history compounds, so trim tool results early.

→ Batch: half price, up to 24 hours, no multi-turn tools, 29-day retention.

→ Attribute by tenant, feature and success. Totals hide the problem.

→ Semantic caches return similar questions' answers. Similarity misses numbers and negation.

→ Compare savings with the cost of a wrong answer. Consider an exact cache or intents.

Your notes